

Appendix B

Cheat-sheet: Elm vs. Haskell

Table B.1: Elm vs. Haskell Basic language constructs and operators

Feature	Elm	Haskell
Single-line comments	--	--
Multi-line comments	Start: {-, End: -}	Start: {-, End: -}
Logical operators	$\wedge$: &&, $\vee$:	$\wedge$: &&, $\vee$:
Equality operator	==, /=	==, /=
Function signature	inc : Int -> Int	inc :: Int -> Int
Function declaration	inc x = x + 1	inc x = x + 1
Local declarations	<pre>double a = let d = 2 * a in d</pre>	<pre>double a = let d = 2 * a in d double a = d where d = 2 * a</pre>

Table B.2: Elm vs. Haskell: Numeric types

Feature	Elm	Haskell
Integer type	<code>Int</code>	<code>Int</code>
Arbitrary precision integers	N/A	<code>Integer</code>
Floating point type	<code>Float</code>	<code>Float</code>
Type constraint for numbers	<code>number</code>	<code>Num</code>
Power operator	<code>^</code>	For integers: <code>^</code> For real numbers: <code>**</code>

Table B.3: Elm vs. Haskell: Lists and Strings

Feature	Elm	Haskell
Lists	<code>1::2::3::[]</code> <code>[1, 2, 3]</code>	<code>1:2:3:[]</code> <code>[1, 2, 3]</code>
List type	<code>List a</code>	<code>[a]</code>
Head and Tail functions	<code>head</code> , <code>tail</code>	<code>head</code> , <code>tail</code>
Characters	<code>'a'</code>	<code>'a'</code>
Strings	<code>"Hello"</code>	<code>"Hello"</code>
List append	<code>[1] ++ [2]</code>	<code>[1] ++ [2]</code>
String append	<code>"Hello, " ++ "there!"</code>	<code>"Hello, " ++ "there!"</code>
String characters	<code>String.toList</code>	<i>Strings in Haskell are defined as</i> <code>type String = [Char]</code>

Table B.4: Elm vs. Haskell: Type declarations

Feature	Elm	Haskell
Lambdas	<code>\x -> x + 1</code>	<code>\x -> x + 1</code>
Curried Lambdas	<code>\x y -> x + y</code>	<code>\x y -> x + y</code>
Tuples	<code>(1, 'a') : (Int, Char)</code>	<code>(1, 'a') :: (Int, Char)</code>
Type declarations	<code>type User = User String String Int</code>	<code>data User = User String String Int</code>
Sum type declarations	<code>type Color = Red Green Blue</code>	<code>data Color = Red Green Blue</code>
Generic data declarations	<code>type Tree a = Nil Node (Tree a) a (Tree a)</code>	<code>data Tree a = Nil Node (Tree a) a (Tree a)</code>
Type aliases	<code>type alias Age = Int</code>	<code>type Age = Int</code>
Type constraints	<code>sum : List number -> number</code>	<code>sum :: Num a => [a] -> a</code>

Table B.5: Elm vs. Haskell: Using the REPL

Feature	Elm	Haskell
Start REPL	<code>elm repl</code>	<code>ghci</code>
View type of function in the REPL	<code>> fnName</code>	<code>> :t fnName</code>
Quit REPL	<code>> :exit</code>	<code>> :quit</code> or <code>> :q</code>
Load module in the REPL	<code>> import ModuleName1 > import ModuleNameN</code>	<code>> :l ModName1 ModNameN</code>
Reload module	Automatic	<code>> :reload</code> or <code>> :r</code>
Multi-line functions	Automatic	Between: <code>{</code> and <code>}</code>
Access last result	N/A	<code>it</code>